Documentación técnica — Sistema de Cálculo Numérico: Método de la Secante

> Material base para el informe técnico del Trabajo Práctico de Matemática IV (Cálculo Numérico). Cada sección sigue el orden que pide la guía del trabajo práctico (puntos 7, 8, 9, 11 y 16). La portada (universidad, facultad, carrera, integrantes, docente, semestre) va en el informe final.

Índice

1. Tecnologías utilizadas
2. Fundamento matemático
3. Diseño del sistema
4. Implementación
5. Validación de datos
6. Casos de prueba y resultados
7. Funcionalidades adicionales (desafío de extensión)
8. Manual de uso

1. Tecnologías utilizadas

Aspecto	Elección	Por qué
Lenguaje de la lógica	**Python 3.12**	Sintaxis clara, ideal para traducir algoritmos matemáticos; permite generar un ejecutable.
Interfaz	**HTML5 + CSS3 + JavaScript** (sin frameworks)	Permite gráficos y animaciones ricas; se abre en cualquier navegador.
Servidor	`http.server` de la **biblioteca estándar** de Python	Conecta la interfaz con la lógica sin instalar nada.
Gráficos	**Canvas 2D** (interfaz) y **SVG** generado en Python (informe)	Dibujados a mano, sin bibliotecas de gráficos.
Ejecutable	**PyInstaller**	Empaqueta Python, el código y la interfaz en un solo `MetodoSecante.exe`.
Pruebas	`unittest` (biblioteca estándar)	42 pruebas automáticas.
IDE	Visual Studio Code	—
Control de versiones	Git + GitHub	—

Aspecto	Elección	Por qué
Tipografías	Archivo y STIX Two Text (licencia SIL OFL)	Incluidas en el proyecto: funciona **sin internet**.

Bibliotecas: qué se usa y qué NO

El proyecto **no usa ninguna biblioteca matemática**: ni `math`, ni `numpy`, ni `scipy`, ni `sympy`, ni `eval()`. Todo el cálculo está implementado desde cero:

Lo que normalmente se importaría	Lo que hicimos nosotros
`math.sin`, `math.cos`, `math.exp`, `math.log`, `math.sqrt`…	Series de Taylor con reducción de argumento y método de Newton (`nucleo/matematica.py`)
`eval("x**3 - 4*x - 1")` o `sympy.sympify`	Tokenizador + parser descendente recursivo + árbol de sintaxis (`nucleo/expresiones.py`)
`sympy.diff` (derivada para Newton-Raphson)	Derivación simbólica con las reglas de derivación sobre el árbol (`nucleo/expresiones.py`)
`scipy.optimize.newton` / `brentq`	Secante, bisección, falsa posición y Newton-Raphson escritos a mano (`nucleo/secante.py`, `nucleo/comparacion.py`)
`matplotlib`	Gráficos en canvas (interfaz) y SVG (informe) dibujados a mano

Solo se usan módulos de la biblioteca estándar para tareas **no matemáticas**: `json` (historial), `http.server` y `socket` (servidor local), `webbrowser` (abrir el navegador), `html.escape` (informe) y `threading`. El módulo `math` aparece **únicamente en las pruebas**, como referencia para verificar que nuestras funciones dan el mismo resultado.

---

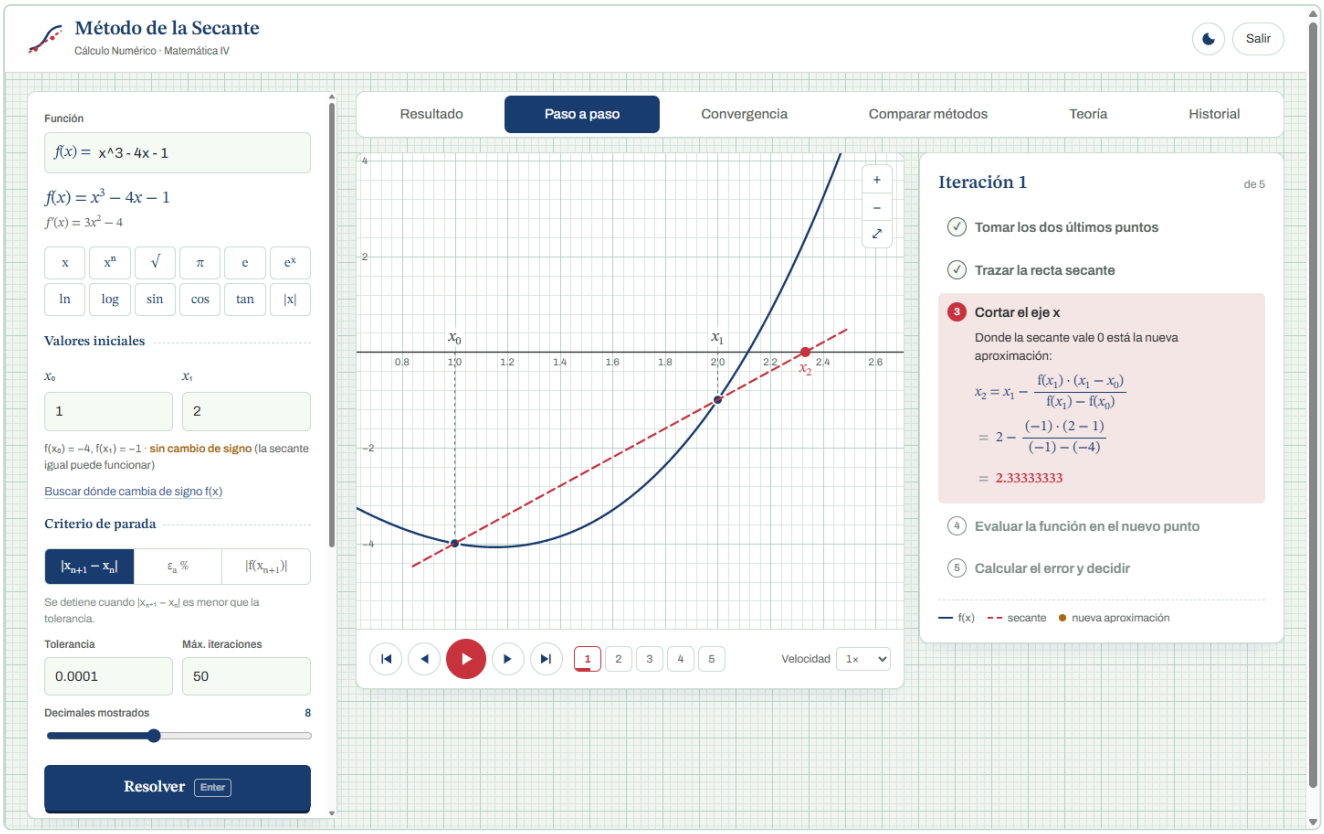
2. Fundamento matemático

2.1 Concepto

El método de la secante resuelve ecuaciones no lineales de la forma **f(x) = 0**. Es un método **abierto e iterativo**.

Parte de **dos aproximaciones iniciales** x_0 y x_1. Por los puntos $(x_0, f(x_0))$ y $(x_1, f(x_1))$ traza una recta **secante** y toma como nueva aproximación el punto donde esa recta corta al eje x. Después repite el proceso con los dos últimos puntos, descartando el más viejo.

Geométricamente, reemplaza la curva por una recta en las cercanías de la raíz. Cuanto más cerca estamos de la raíz, más se parece la curva a esa recta y mejor es la aproximación.



2.2 Fórmula

La recta que pasa por $(x_{n-1}, f(x_{n-1}))$ y $(x_n, f(x_n))$ tiene pendiente

$$m = \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

y su ecuación es $y = f(x_n) + m \cdot (x - x_n)$.

Para hallar el corte con el eje x se hace $y = 0$ y se despeja x:

$$x_{n+1} = x_n - \frac{f(x_n) \cdot (x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

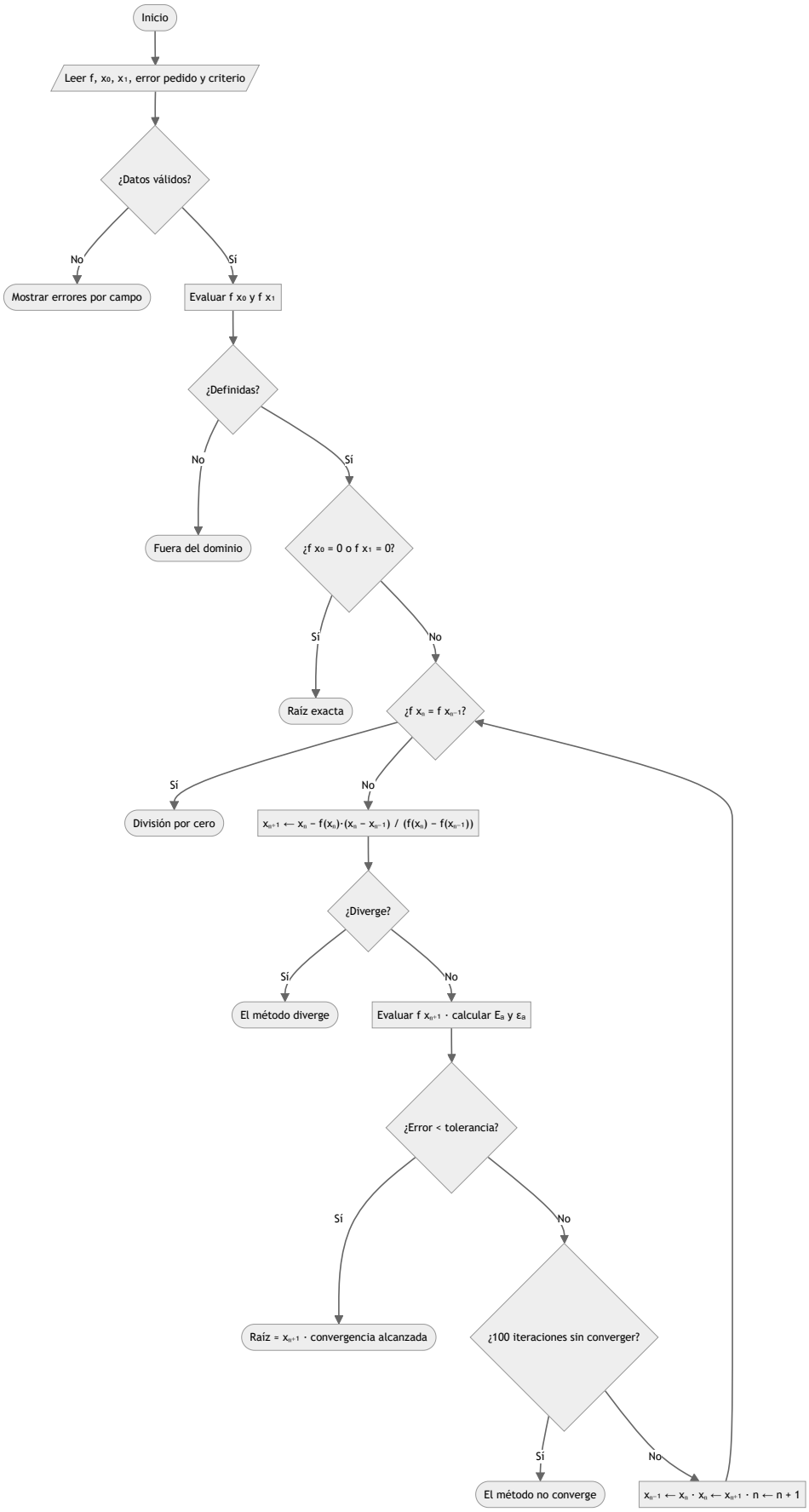
Relación con Newton-Raphson. La fórmula de Newton es $x_{n+1} = x_n - f(x_n)/f'(x_n)$. La secante reemplaza la derivada $f'(x_n)$ por el cociente incremental m, que es la pendiente de la secante. Por eso **no necesita conocer la derivada**.

2.3 Algoritmo

```
Entrada: f, x₀, x₁, error pedido (tolerancia), criterio de parada
Salida:  raíz aproximada, tabla de iteraciones, estado final

 1. Calcular f(x₀) y f(x₁). Si alguno no está definido → informar "fuera del dominio".
 2. Si f(x₀) = 0 o f(x₁) = 0 → ese valor es la raíz exacta.
 3. Para n = 1, 2, 3, … (hasta alcanzar el error pedido):
 4.     Si f(xₙ) − f(xₙ₋₁) = 0 → detener: la secante es horizontal (división por cero).
 5.     xₙ₊₁ ← xₙ − f(xₙ)·(xₙ − xₙ₋₁) / (f(xₙ) − f(xₙ₋₁))
 6.     Si xₙ₊₁ es infinito o crece sin control → detener: el método diverge.
 7.     Calcular f(xₙ₊₁). Si no está definido → detener: fuera del dominio.
 8.     Eₐ ← |xₙ₊₁ − xₙ|          εₐ ← |Eₐ / xₙ₊₁| · 100 %
 9.     Guardar la fila de la tabla.
10.     Si f(xₙ₊₁) = 0 → raíz exacta.
11.     Si el error elegido < tolerancia → convergencia alcanzada.
12.     xₙ₋₁ ← xₙ ,  xₙ ← xₙ₊₁       (se descarta el punto más viejo)
13. Protección: si tras 100 iteraciones el error sigue sin bajar → informar que no converge.
```

Diagrama de flujo


Inicio
Leer f, x_0, x_1, error pedido y criterio
¿Datos válidos?
No
Sí
Mostrar errores por campo
Evaluar f x_0 y f x_1
¿Definidas?
No
Sí
Fuera del dominio
¿f x_0 = 0 o f x_1 = 0?
Sí
No
Raíz exacta
¿f x_n = f x_{n-1}?
Sí
No
División por cero
$x_{n+1} \leftarrow x_n - f(x_n)\cdot(x_n - x_{n-1}) \,/\, (f(x_n) - f(x_{n-1}))$
¿Diverge?
Sí
No
El método diverge
Evaluar f x_{n+1} · calcular E_a y ε_a
¿Error < tolerancia?
Sí
No
Raíz = x_{n+1} · convergencia alcanzada
¿100 iteraciones sin converger?
Sí
No
El método no converge
$x_{n-1} \leftarrow x_n$ · $x_n \leftarrow x_{n+1}$ · $n \leftarrow n + 1$

2.4 Condiciones de aplicación

- **f debe ser continua** cerca de la raíz y, para que converja con el orden esperado, **derivable con f′(r) ≠ 0** y f″ continua.
- Se necesitan **dos valores iniciales distintos** ($x_0 \neq x_1$).
- **No** es necesario que x_0 y x_1 encierren la raíz: $f(x_0)$ y $f(x_1)$ pueden tener el mismo signo. Esa es la diferencia principal con bisección y falsa posición.
- Conviene que x_0 y x_1 estén **cerca de la raíz**. Si $f(x_0)\cdot f(x_1) < 0$ y f es continua, el **teorema de Bolzano** garantiza al menos una raíz entre ambos. El sistema lo verifica e informa el resultado, pero no lo exige.

2.5 Criterio de convergencia (orden φ ≈ 1.618)

Sea $e_n = x_n - r$ el error en el paso n. Desarrollando f por Taylor alrededor de la raíz r se obtiene

$$e_{n+1} \approx C \cdot e_n \cdot e_{n-1} \qquad \text{con} \quad C = f''(r) / (2 \cdot f'(r))$$

Si buscamos un orden p tal que $|e_{n+1}| \approx A \cdot |e_n|^p$, al reemplazar resulta

$$p^2 = p + 1 \quad \Longrightarrow \quad p = (1 + \sqrt{5}) / 2 = \phi \approx 1.618 \qquad \text{(el número áureo)}$$

- La convergencia es **superlineal**: más rápida que bisección (lineal, p = 1) y algo más lenta que Newton (cuadrática, p = 2).
- En la práctica, **la cantidad de cifras correctas se multiplica por ≈ 1.6 en cada iteración**.
- Si la raíz es **múltiple** ($f'(r) = 0$), la convergencia pasa a ser solo lineal. El sistema lo detecta: con $f(x) = (x - 1)^2$ estima p = 1.000.
- El sistema **estima p experimentalmente** a partir de tres errores consecutivos:

$$p \approx \frac{\ln(e_{n+1} / e_n)}{\ln(e_n / e_{n-1})}$$



2.6 Criterio de parada

El usuario elige uno de tres criterios:

Criterio	Condición para detenerse	Uso típico
Error absoluto	$\lvert x_{n+1} - x_n \rvert <$ tolerancia	Cuando importa la precisión en x (por ejemplo, 4 decimales).
Error relativo porcentual	$\lvert (x_{n+1} - x_n)/x_{n+1} \rvert \cdot 100 <$ tolerancia (en %)	Cuando la raíz es muy grande o muy chica.
Residuo	$\lvert f(x_{n+1}) \rvert <$ tolerancia	Cuando importa que f quede cerca de 0.

Además, el método **siempre** se detiene:

- al encontrar $f(x_{n+1}) = 0$ exactamente (raíz exacta);
- **no hay un máximo de iteraciones elegido por el usuario**: se itera hasta alcanzar el error pedido. Solo existe un límite interno de seguridad de 100 iteraciones, que actúa únicamente cuando el método **no converge** (sin él, el programa quedaría en un ciclo infinito). Cuando converge, la secante lo hace en pocas iteraciones;
- ante **división por cero**, **divergencia** o **salida del dominio**, informando el motivo;
- si las aproximaciones dejan de cambiar porque se llegó al límite de precisión de la máquina.

2.7 Error aproximado

Como no conocemos la raíz exacta, el error se **aproxima** con la diferencia entre dos aproximaciones consecutivas:

```
Error absoluto aproximado:   E_a = |x_{n+1} − x_n|
Error relativo aproximado:   ε_a = |(x_{n+1} − x_n) / x_{n+1}| · 100 %
```

Con el **criterio de Scarborough** el sistema informa cuántas cifras significativas son confiables. Si $\varepsilon_a < 0.5 \cdot 10^{(2-n)}$ %, el resultado tiene al menos n cifras correctas. Despejando, $n = 2 - \log_{10}(2 \cdot \varepsilon_a)$.

Otras fuentes de error que el sistema contempla:

- **Error de redondeo.** La computadora trabaja con unas 16 cifras significativas (doble precisión IEEE 754). Por eso no se aceptan tolerancias menores a 10^{-15}.
- **Error de cancelación.** Cerca de la raíz, $f(x_n) - f(x_{n-1})$ es una resta de números muy parecidos. Por esa razón las estimaciones del orden que ya están dominadas por el redondeo se descartan.

2.8 Ventajas

1. **No necesita la derivada**, que puede ser difícil o imposible de obtener (por ejemplo, si f sale de una simulación).
2. Converge rápido, con orden ≈ 1.618: en el ejemplo del enunciado necesita 6 iteraciones contra 27 de bisección.
3. Hace **una sola evaluación nueva de f por iteración**, mientras que Newton necesita f y f′. En costo por evaluación, muchas veces es más eficiente que Newton.
4. No exige un intervalo con cambio de signo.

2.9 Limitaciones

1. **No garantiza convergencia**: puede diverger si los valores iniciales están lejos de la raíz. Ejemplo: $f(x) = 1/x$.
2. **División por cero** si $f(x_n) = f(x_{n-1})$: la secante es horizontal. Ejemplo: $f(x) = x^2 - 4$ con $x_0 = -1$, $x_1 = 1$.
3. Como **no encierra la raíz**, puede saltar a otra raíz distinta de la buscada o salir del dominio de f. Ejemplo: $\ln(x)$ con $x_0 = -1$.
4. En **raíces múltiples** la convergencia se vuelve lineal y lenta.
5. Sufre **error de cancelación** cerca de la raíz.

2.10 Métodos complementarios (para la comparación)

Método	Tipo	Fórmula	Orden	Requiere
Secante	Abierto	$x_{n+1} = x_n - f(x_n)(x_n - x_{n-1})/(f(x_n) - f(x_{n-1}))$	$\varphi \approx 1.618$	2 valores iniciales
Newton-Raphson	Abierto	$x_{n+1} = x_n - f(x_n)/f'(x_n)$	2	$f'(x)$ (calculada simbólicamente por el sistema)
Falsa posición	Cerrado	$c = b - f(b)(b - a)/(f(b) - f(a))$	1 (lineal)	Cambio de signo en [a, b]
Bisección	Cerrado	$c = (a + b)/2$	1 (lineal)	Cambio de signo en [a, b]

La **falsa posición** usa la misma fórmula que la secante. La diferencia es que conserva el extremo que mantiene el cambio de signo, en lugar de descartar siempre el punto más viejo. Por eso siempre converge,

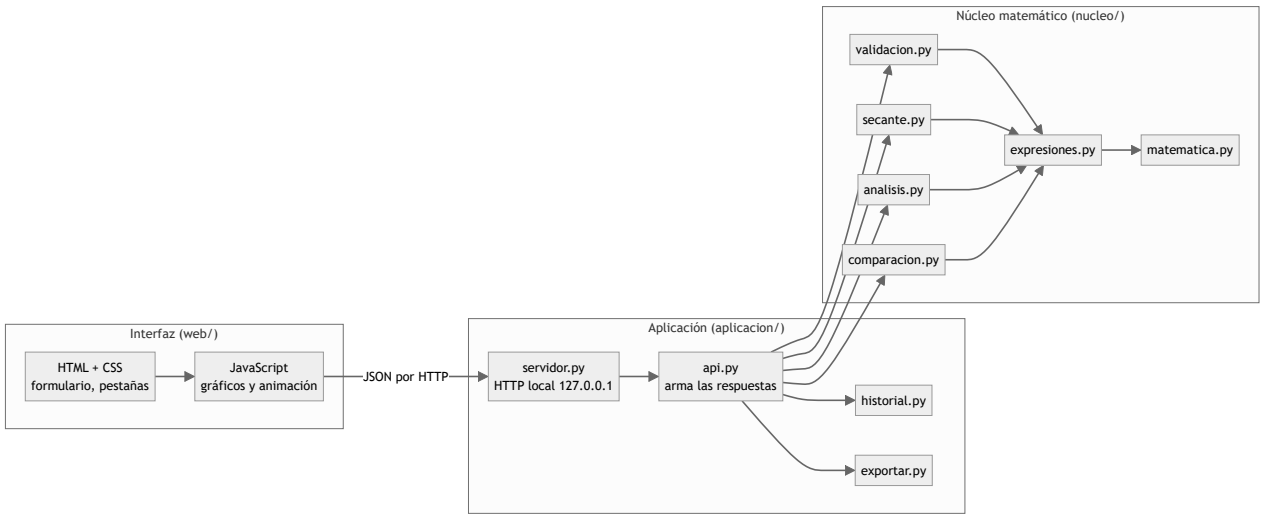
aunque más lento.

---

3. Diseño del sistema

3.1 Arquitectura general

El sistema tiene tres capas. La **lógica matemática** es independiente de la interfaz: se puede probar y usar sin el navegador.



Flujo de un cálculo: el usuario completa el formulario → JavaScript envía los datos en JSON a `/api/resolver` → `validacion.py` controla cada campo → `expresiones.py` convierte el texto de la función en un árbol evaluable → `secante.py` itera → `analisis.py` y `comparacion.py` estudian el resultado → la respuesta vuelve en JSON → la interfaz la muestra y la anima.

3.2 Estructura de módulos

```
MATEMATICA/
├── app.py                    Punto de entrada: levanta el servidor y abre el navegador
├── construir_exe.py          Genera dist/MetodoSecante.exe con PyInstaller
├── nucleo/                   ── LÓGICA MATEMÁTICA (sin bibliotecas) ──
│   ├── matematica.py         exp, ln, sen, cos, atan, √, potencias… (series de Taylor, Newton)
│   ├── expresiones.py        Intérprete de funciones: tokenizador, parser, evaluación,
derivada
│   ├── secante.py            ★ Método de la secante
│   ├── validacion.py         Validación de los datos de entrada
│   ├── analisis.py           Bolzano, búsqueda de intervalos, orden de convergencia,
conclusiones
│   └── comparacion.py        Bisección, falsa posición y Newton-Raphson para comparar
├── aplicacion/               ── CONEXIÓN CON LA INTERFAZ ──
│   ├── api.py                Funciones que atiende el servidor
│   ├── servidor.py           Servidor HTTP local (biblioteca estándar)
│   ├── historial.py          Historial de ejercicios en JSON
│   └── exportar.py           Exportación a CSV, TXT e informe HTML con gráfico SVG
├── web/                      ── INTERFAZ ──
│   ├── index.html, css/estilos.css
│   ├── js/                   app.js, lienzo.js (gráfico), pasos.js (animación), lineas.js, …
│   └── fuentes/              Tipografías incluidas (funciona sin internet)
├── tests/                    42 pruebas automáticas
└── docs/                     Esta documentación, la guía del código y las capturas
```

3.3 Pseudocódigo del intérprete de funciones

El texto `"x^3 - 4x - 1"` se procesa en tres etapas.

```
1) TOKENIZAR:   "x^3 - 4x - 1"  →  [x] [^] [3] [−] [4] [x] [−] [1]

2) PARSEAR (gramática, de menor a mayor precedencia):
     expresion → termino (('+' | '−') termino)*
     termino   → unario (('*' | '/') unario | multiplicación implícita)*
     unario    → '−' unario | potencia
     potencia  → primario ('^' unario)?
     primario  → número | x | constante | función '(' expresion ')' | '(' expresion ')'

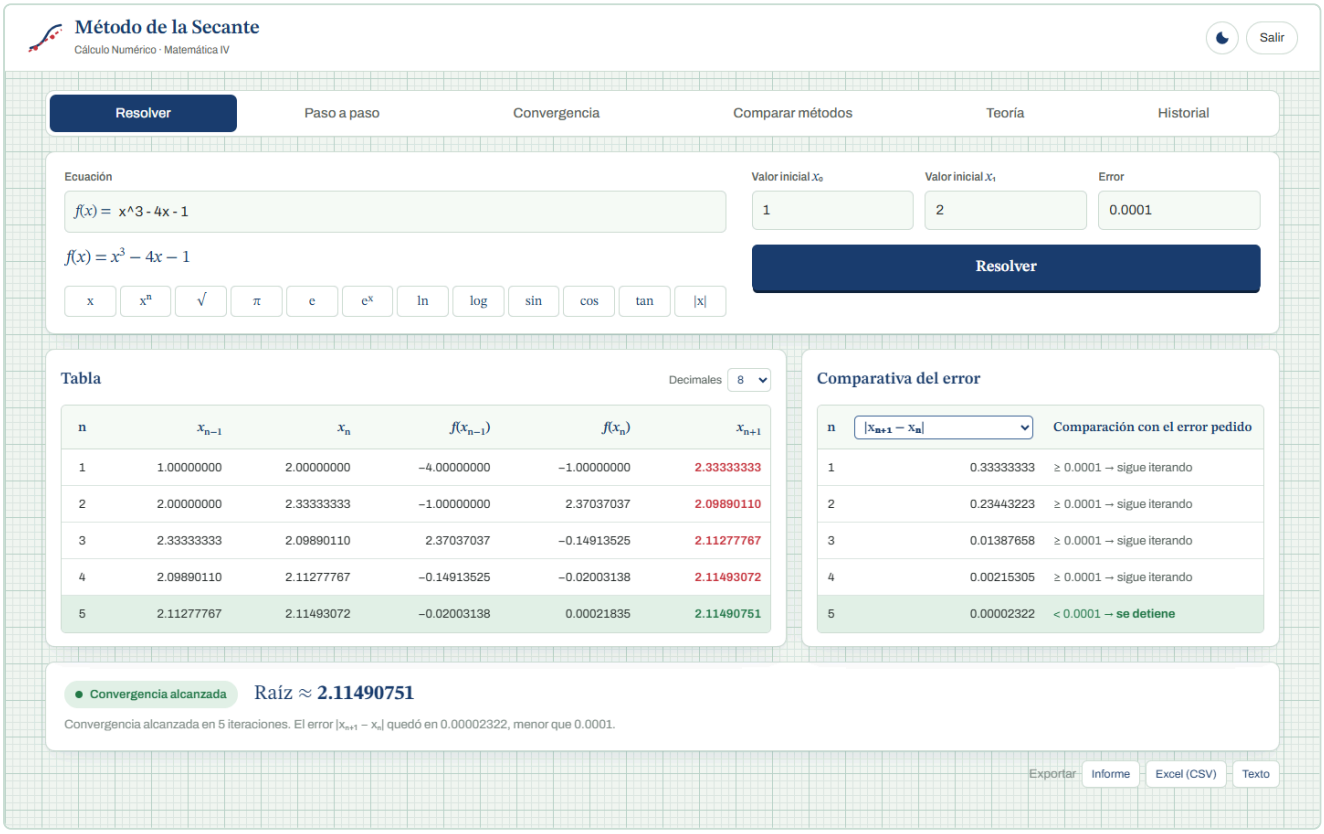
   Resultado: árbol de sintaxis
                 (−)
                /   \
            (−)     1
           /   \
         (^)    (·)
        /  \    /  \
       x    3  4    x

3) EVALUAR: se recorre el árbol; cada nodo calcula su valor con matematica.py
```

3.4 Diseño de la interfaz

La interfaz usa la estética de una **hoja de papel milimetrado de dibujo técnico**: tinta azul para la función y lápiz rojo para las rectas secantes. Tiene modo claro y modo oscuro (estilo cianotipo).

Zona	Contenido
Resolver (página de inicio)	Arriba, los datos: la ecuación f(x) con vista previa y teclado matemático, los valores iniciales x0 y x1 y el error pedido. Abajo, dos cuadros lado a lado: a la izquierda **la tabla** (n, x_{n-1}, x_n, $f(x_{n-1})$, $f(x_n)$, x_{n+1}) que arranca con los valores iniciales; a la derecha **la comparativa del error**, fila por fila, con el **criterio de parada elegible desde la cabecera del cuadro**. Debajo, la raíz obtenida y la exportación.
Paso a paso	Animación de cada iteración en 5 subpasos con la fórmula sustituida con los números reales.
Convergencia	Error por iteración (escala logarítmica) y orden de convergencia estimado comparado con φ.
Comparar métodos	Secante, Newton-Raphson, falsa posición y bisección sobre el mismo problema.
Teoría	Fundamento, fórmula, algoritmo, condiciones, ventajas y limitaciones.
Historial	Ejercicios resueltos, con la opción de volver a resolverlos.



4. Implementación

4.1 El método de la secante (`nucleo/secante.py`)

Fragmento central del bucle de iteraciones:

```
for n in range(1, limite_iteraciones + 1):     # sin máximo del usuario: límite de seguridad

    denominador = f_actual - f_anterior
    if denominador == 0:
        # La secante es horizontal: nunca corta al eje x
        resultado.estado = DIVISION_CERO
        ...
        return resultado

    # Fórmula de la secante
    x_siguiente = x_actual - f_actual * (x_actual - x_anterior) / denominador

    if not m.es_finito(x_siguiente) or m.valor_absoluto(x_siguiente) > limite:
        resultado.estado = DIVERGENCIA
        ...

    f_siguiente, error = _evaluar(f, x_siguiente, "x%d" % (n + 1))
    error_absoluto, error_relativo = calcular_errores(x_siguiente, x_actual)
    resultado.iteraciones.append(Iteracion(n, x_anterior, x_actual, f_anterior, f_actual,
                                           x_siguiente, f_siguiente, error_absoluto,
error_relativo))

    if cumple_criterio(criterio, tolerancia, error_absoluto, error_relativo, f_siguiente):
        resultado.estado = CONVERGIO
        return resultado

    # Se descarta el punto más viejo y se avanza
    x_anterior, f_anterior = x_actual, f_actual
    x_actual, f_actual = x_siguiente, f_siguiente
```

4.2 Funciones matemáticas propias (`nucleo/matematica.py`)

Ejemplo: $\mathbf{e^x}$ por reducción de argumento y serie de Taylor.

```
def exp(x):
    # x = k·ln2 + r  con |r| ≤ 0.347   ⟹   eˣ = 2ᵏ · eʳ
    k = redondear_entero(x / LN2)
    r = (x - k * LN2_ALTO) - k * LN2_BAJO

    suma = 1.0
    termino = 1.0
    for n in range(1, MAX_TERMINOS):
        termino *= r / n               # rⁿ/n! a partir del término anterior
        nueva = suma + termino
        if nueva == suma:              # el término ya no cambia el resultado
            break
        suma = nueva
    return escalar_por_potencia_de_dos(suma, k)
```

Función	Técnica
e^x	Reducción $x = k \cdot \ln 2 + r$ y serie de Taylor de e^r
ln x	$x = m \cdot 2^k$ y serie de la arcotangente hiperbólica: $\ln m = 2 \cdot \operatorname{atanh}((m-1)/(m+1))$
sen x, cos x	Reducción al intervalo $[-\pi/4, \pi/4]$ (cuadrantes) y series de Taylor
atan x	Identidades $\operatorname{atan}(x) = \pi/2 - \operatorname{atan}(1/x)$ y desplazamiento de $\pi/6$, luego serie
asin, acos	A partir de atan y √
√x	Método de Newton: $y_{n+1} = (y_n + x/y_n)/2$
$\sqrt[3]{x}$	Aproximación con exp/ln y pulido con Newton
a^b	Exponenciación binaria si b es entero; e^(b·ln a) si no

Todas coinciden con el módulo `math` de Python con un error relativo menor a 10^{-12} (lo verifican las pruebas).

4.3 Derivación simbólica (`nucleo/expresiones.py`)

Para comparar con Newton-Raphson hace falta f′(x). En lugar de usar una biblioteca, el sistema **deriva el árbol** aplicando las reglas de derivación y después simplifica el resultado:

```
if op == "*":
    if not contiene_x(u):
        return Binario("*", u, dv)                                  # (c·v)' = c·v'
    return Binario("+", Binario("*", du, v), Binario("*", u, dv))  # regla del producto
```

Ejemplo: para $f(x) = x^3 - 4x - 1$ el sistema obtiene $f'(x) = 3x^2 - 4$.

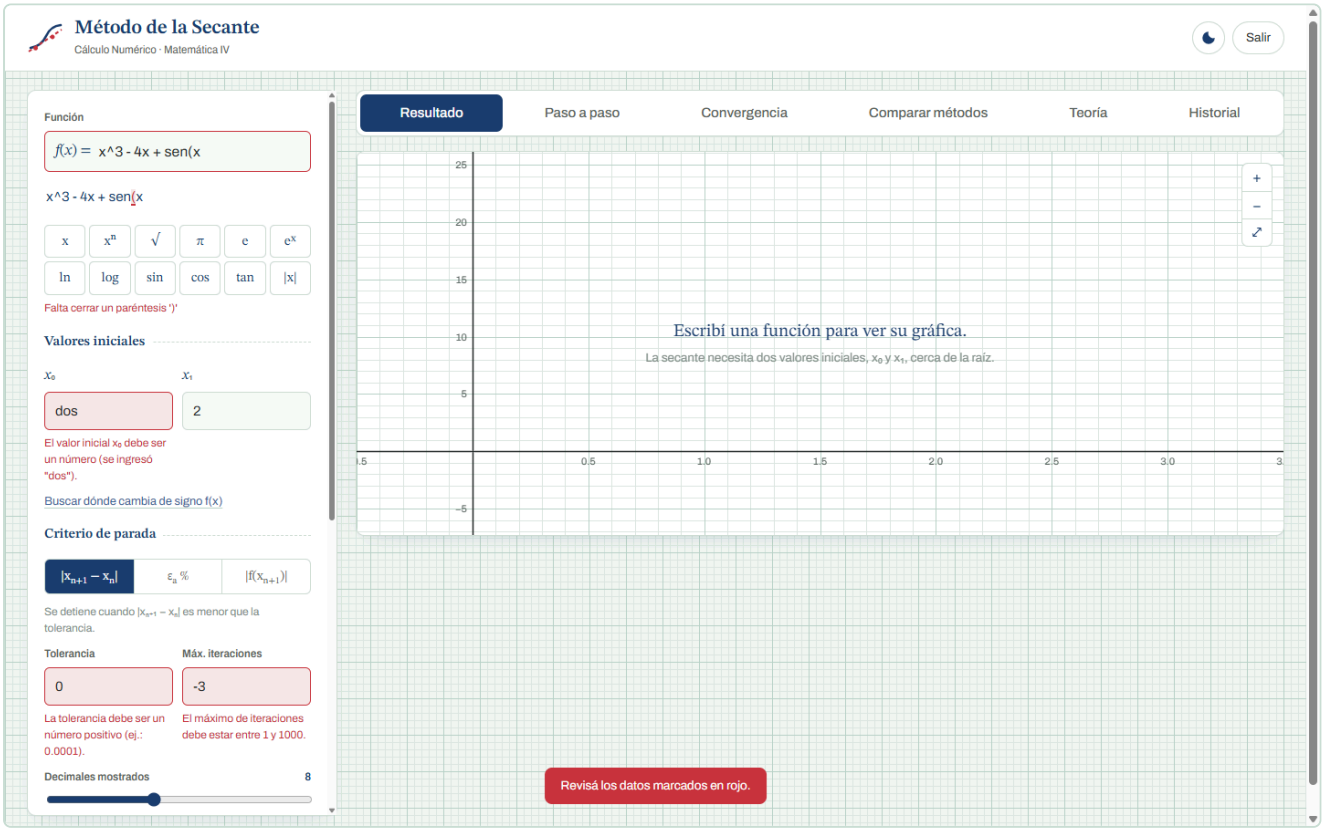
---

5. Validación de datos

Correspondencia con el punto 9 de la guía del trabajo práctico. El sistema **nunca se cierra inesperadamente**: cada problema se informa con un mensaje claro junto al campo correspondiente.

Situación (guía)	Ejemplo	Mensaje del sistema
Datos incompletos	x_0 vacío	"Falta ingresar el valor inicial x_0."
Valores no numéricos	x_0 = "abc"	"El valor inicial x_0 debe ser un número (se ingresó "abc")."
Función mal escrita	`x^3 - 4x + sen(x`	"Falta cerrar un paréntesis ')'" (con el lugar subrayado en rojo)
Variable desconocida	`x + y`	"No se reconoce 'y'. La función debe depender solo de x..."
Intervalo incorrecto	$x_0 = x_1$	"x_0 y x_1 deben ser distintos: la secante necesita dos puntos diferentes..."
División por cero	$f(x_n) = f(x_{n-1})$	"La recta secante es horizontal y no corta al eje x. Probá con otros valores iniciales."
División por cero en f	$1/x$ en $x = 0$	"f(x) no se puede evaluar en ... División por cero"
Fuera del dominio	$\ln(x)$ con $x_0 = -1$	"√x solo está definida para $x \geq 0$...", "ln(x) solo está definido para $x > 0$..."
Tolerancia inválida	0, −1 o 10^{-20}	"La tolerancia debe ser un número positivo..." / "...demasiado chica..."
Condiciones que impiden la convergencia	$1/x$, $x^2 + 1$	"El método diverge..." / "Después de 100 iteraciones el error todavía no es menor al pedido: el método no converge..."
Función constante	$f(x) = 5$	"La función es constante (no depende de x)..."
Coma decimal	0,001	Se acepta y se convierte a 0.001
Valores como expresión	x_0 = pi/4	Se acepta: los campos numéricos admiten expresiones constantes

Además, el sistema **junta todos los errores a la vez** (no solo el primero) y los marca en rojo en cada campo.



6. Casos de prueba y resultados

6.1 Ejemplo del enunciado, verificado a mano

f(x) = $x^3 - 4x - 1$, $x_0 = 1$, $x_1 = 2$, tolerancia 0.0001 (error absoluto)

Iteración 1, a mano: - f(1) = 1 − 4 − 1 = −4 - f(2) = 8 − 8 − 1 = −1 - $x_2 = 2 - (-1)\cdot(2 - 1) / (-1 - (-4)) = 2 + 1/3$ = **2.33333333** ✓ (coincide con el sistema)

n	x_{n-1}	x_n	x_{n+1}	$f(x_{n+1})$	E_a	ε_a %
1	1.00000000	2.00000000	2.33333333	2.37037037	0.33333333	14.28571429
2	2.00000000	2.33333333	2.09890110	−0.14913525	0.23443223	11.16928447
3	2.33333333	2.09890110	2.11277767	−0.02003138	0.01387658	0.65679298
4	2.09890110	2.11277767	2.11493072	0.00021835	0.00215305	0.10180237
5	2.11277767	2.11493072	2.11490751	$-3.1362\cdot10^{-7}$	0.00002322	0.00109772

Resultado: raíz aproximada **2.114907508**, 5 iteraciones, error aproximado $2.32\cdot10^{-5}$, estado: convergencia alcanzada. Con una tolerancia más chica la raíz da 2.11490754148, que coincide con la raíz real de $x^3 - 4x - 1 = 0$.

6.2 Batería de casos

Tolerancia y criterio indicados; máximo 50 iteraciones. Las columnas de la derecha son la comparación automática.

f(x)	x_0, x_1	Tol.	Estado	Iter.	Raíz	p estimado	Newton	Falsa pos.	Bisección
$x^3 - 4x - 1$	1, 2	10^{-4}	Convergió	5	2.114907508	1.545	3	no aplica	no aplica
$x^3 - 4x - 1$	2, 3	10^{-8}	Convergió	6	2.114907541	1.525	6	19	27
$x^3 - 4x - 1$	−1, 0	10^{-4}	Convergió	4	−0.254101722	1.716	3	4	14
cos x − x	0, 1	10^{-8}	Raíz exacta*	6	0.739085133	1.599	4	8	27
$e^{-x} - x$	0, 1	10^{-8}	Convergió	6	0.567143290	1.667	4	9	27
$x^2 - 2$	1, 2	10^{-10}	Convergió	7	1.414213562 (= √2)	1.579	5	14	34
x·sen x − 1	0, 2	10^{-8}	Convergió	5	1.114157141	0.688†	6	5	28
ln x + x − 2	1, 3	10^{-8}	Raíz exacta*	6	1.557145599	1.569	5	9	28

f(x)	x_0, x_1	Tol.	Estado	Iter.	Raíz	p estimado	Newton	Falsa pos.	Bisección
$(x-1)^2$	0, 0.5	10^{-4}	Convergió (lento)	17	0.999852	**1.000**	13	no aplica	no aplica
x^2+1	0, 1	10^{-4}	División por cero	—	sin raíz real	—	—	—	—
x^2-4	−1, 1	10^{-4}	División por cero	—	—	—	—	—	—
1/x	−1, 2	10^{-4}	Diverge	—	sin raíz	—	—	—	discontinuidad**
ln x	−1, 2	10^{-4}	Fuera del dominio	—	—	—	—	—	—

* "Raíz exacta" significa que $f(x_{n+1})$ dio exactamente 0 en la aritmética de la computadora. † Con pocas iteraciones, y partiendo lejos de la raíz, la estimación de p no es confiable: la fórmula supone que ya estamos cerca de r. ** La bisección se estabiliza en x = 0, pero ahí f(x) no se anula: es una asíntota. El sistema lo detecta y no lo cuenta como raíz.

Análisis: - En los casos de raíz simple con suficientes iteraciones, el orden estimado está entre 1.52 y 1.72, muy cerca del teórico $\varphi \approx 1.618$. - En la raíz doble $(x-1)^2$ el orden baja a 1.000: se confirma la limitación teórica. - La secante necesitó entre 3 y 5 veces menos iteraciones que la bisección. - En el primer caso, bisección y falsa posición **no se pueden aplicar** (no hay cambio de signo en [1, 2]), pero la secante igual encuentra la raíz: es la ventaja de un método abierto.

6.3 Verificación con herramientas de referencia

La carpeta `tests/` contiene **42 pruebas automáticas** (`python -m unittest discover -s tests -t .`) que verifican:

- Las funciones matemáticas propias contra el módulo `math`, en miles de puntos (error relativo < 10^{-12}).
- El parser: precedencia, multiplicación implícita y errores de sintaxis con su posición.
- La derivada simbólica contra el cociente incremental numérico.
- La primera iteración del ejemplo, calculada a mano.
- Raíces conocidas (√2, el número omega 0.5671..., el número de Dottie 0.7390...).
- Todos los estados de error: división por cero, divergencia, dominio y falta de convergencia.
- La validación de cada tipo de dato inválido.
- El orden de convergencia ≈ φ y la detección de asíntotas.

---

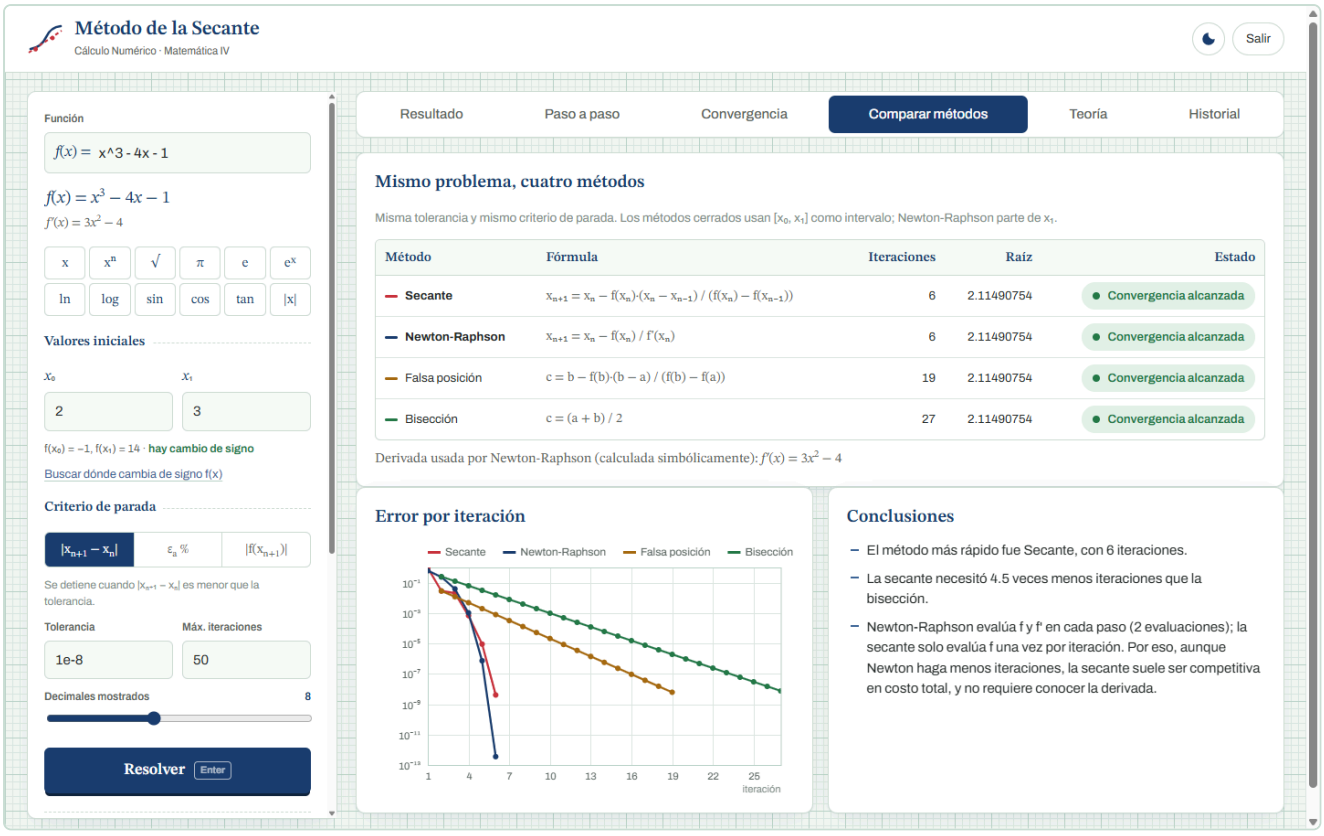
7. Funcionalidades adicionales (desafío de extensión)

Correspondencia con el punto 16 de la guía del trabajo práctico:

Funcionalidad sugerida	Implementación
Representación gráfica de funciones	Gráfico interactivo en el paso a paso, con zoom, desplazamiento, coordenadas y asíntotas. Vista previa matemática de la ecuación mientras se escribe.
Gráficas de convergencia	Error por iteración (escala logarítmica) y orden de convergencia estimado.
Comparación automática entre métodos	Secante contra Newton-Raphson, falsa posición y bisección, con tabla, gráfico y conclusiones.
Cálculo y visualización del error	Error absoluto, relativo y cifras significativas (Scarborough). En la animación, el error se dibuja como una cota de plano técnico.
Exportación de resultados	CSV (se abre en Excel), TXT e informe HTML imprimible como PDF, con gráfico.
Historial de ejercicios resueltos	Se guarda automáticamente y permite volver a resolver o borrar.
Interfaz gráfica avanzada	Animación paso a paso, modo claro y oscuro, adaptable a celular, teclado matemático.
Generación de informes	Informe completo: problema, resultado, Bolzano, gráfico, tabla, análisis y comparación.
Incorporación de métodos adicionales	Bisección, falsa posición y Newton-Raphson (con derivada simbólica propia).
Análisis automático de la convergencia	Orden estimado contra φ, monotonía del error, raíces múltiples, asíntotas, saltos a otras raíces y conclusiones en texto.

Extras no pedidos:

- Verificación del teorema de Bolzano y detección de asíntotas (en el análisis y en el informe exportado).
- Funciones matemáticas propias, sin la biblioteca `math`.
- Funciona sin internet y sin instalar nada (ejecutable).



8. Manual de uso

Ejecutar

- **Ejecutable:** doble clic en `MetodoSecante.exe`. Se abre una ventana negra (el servidor) y el navegador con el programa. Para salir, usá el botón **Salir** o cerrá la ventana negra.
- **Desde el código:** `python app.py` (requiere Python 3.10 o superior, sin dependencias).
- **Generar el ejecutable:** `python -m pip install pyinstaller` y luego `python construir_exe.py`.

Resolver un problema

1. Escribir la función en **f(x) =**. Se aceptan `x^3 - 4x - 1`, `cos(x) - x`, `e^(-x) - x`, `2sin(x)`, `sqrt(x)`, `ln(x)`, `log(x)`, `pi`, `e`, etc. La multiplicación puede omitirse (`4x`).
2. Ingresar los valores iniciales **x0** y **x1**.
3. Ingresar el **error** pedido (por ejemplo 0.0001).
4. Presionar **Resolver** (o Enter). Aparecen la **tabla** y, a la derecha, la **comparativa del error**.
5. Para cambiar el **criterio de parada**, elegirlo en la cabecera del cuadro del error: el problema se vuelve a resolver solo.
6. Opcional: recorrer las pestañas **Paso a paso** (espacio = reproducir o pausar, flechas = avanzar o retroceder), **Convergencia**, **Comparar métodos**, **Teoría** e **Historial**.
7. Exportar con los botones **Informe**, **Excel (CSV)** o **Texto**.